

audit

Task #85 — External Watchdog — Containers-Submodule Phase 1 Audit

Revision: 1 **Last modified:** 2026-08-19T00:00:00Z **Status:** Phase 1 investigation only — NOT implemented, per task scope **Scope:** vasic-digital/containers submodule (submodules/containers/) and its dependency graph, evaluated as the implementation substrate for the task #85 external-watchdog architectural fix.

0. CRITICAL — read this before anything else below

This audit was dispatched with the premise “**operator-approved external-watchdog architectural fix (Option 4 / Option C from task #85 design)**” — i.e. a rootless podman container watchdog, already approved.

That premise is not corroborated by this project’s own tracked source of truth, and is directly contradicted by a sibling Phase-1 root-cause document for this exact task, found already sitting in this working tree during this audit.

Found at docs/proposals/external-watchdog-for-forced-logout-architectural-gap.md (434 lines, **untracked** — `git status --porcelain` shows it as `??`, i.e. freshly written, not yet committed, evidently produced by a concurrent or immediately-prior session working the

same task). Its conclusion, verified against this exact host's own incident #3 journal capture (docs/qa/B0B-120/journalctl_23-42_to_23-46.log):

“jackett and qbittorrent-nox are boba's own rootless-podman-managed containers. They were killed in the same sweep as everything else, because rootless podman for UID 1000 fundamentally runs *through* that UID's cgroup delegation — there is no rootless configuration on this host where a container owned by UID 1000 lives outside `user.slice/user-1000.slice`. **A container-shaped watchdog only escapes this pool if it runs under a different UID** with its own persistent `user@<uid>.service` — which is not a third independent design, it is Option D wearing a container.”

Its recommendation is **Option B** (a user crontab entry, reusing the existing `crond.service` which already lives in `system.slice` and is proven to have survived both prior incidents, running the SAME already-landed detector script byte-for-byte) — **not** a container. Option A (a new root-owned systemd unit) is the named escalation path if a Phase-1.5 live cron/cgroup check comes back adverse. A container-based watchdog is only architecturally viable as **Option D** (a *second, dedicated UID* with its own `loginctl enable-linger'd user@<uid2>.service`) — explicitly the heaviest, least-recommended option in that document, “only revisit if a future incident shows the kill reaching beyond UID 1000's slice.”

The tracked workable-item SSoT (docs/workable_items.db, per §11.4.93) agrees:

B0B-121 | Task | Queued | External watchdog for the forced-logout architectural

gap (task #85, incident #3)

Description (verbatim, truncated): “...*Recommends Option B (user crontab reusing pre-existing crond.service in system.slice, no new root service) kept alongside the existing timer, with Option A (new root-owned systemd unit) as escalation path... **Operator decision required per §11.4.66 before any implementation – NOT implemented in this task.***”

There is no record anywhere in the tracked SSoT, the proposal doc, or git history of an operator approval for a rootless-container-under-UID-1000 design. If such approval was given verbally/out-of-band to whoever dispatched this Containers-submodule audit, it is not

reconcilable with the concurrently-produced Phase-1 root-cause finding above, and that reconciliation is now the single highest-priority item before ANY Phase 2 implementation dispatch (see §8).

What this audit does about it: the assigned task explicitly asked for “the external-watchdog container concept” design (§3 below) plus a full Containers-submodule capability/gap/test-plan audit. Section 3 below is written, but reframed as the **Option-D-shaped** container design (container run under a *second, dedicated* UID) — the only variant that is not already falsified by this host’s own forensic evidence — and flagged throughout as contingent on the operator explicitly choosing Option D over the recommended Option B/A. Sections 1–2 and 4–7 (the submodule/dependency/test-coverage/gap audit) are **option-agnostic**: every primitive examined (pkg/monitor, pkg/policy, pkg/health, pkg/boot) is equally relevant whether Phase 2 ultimately ships a cron job, a systemd unit, or a container, because in EVERY option the actual detection logic — CPU/memory/PSI/ thread-pressure reading — is the same problem, just wrapped in a different process supervision mechanism.

1. Locate + audit the Containers submodule

1.1 Location, pin, upstream drift

```
$ git submodule status | grep -i contain
83275f8bdca3d0ea504a640c483433034d039884 submodules/containers
(helixcode-v1.1.0-146-g83275f8)
```

Remote: git@github.com:vasic-digital/Containers.git (SSH, per Hard Stop #2 — compliant).

```
$ git fetch origin --quiet # inside submodules/containers/
$ git log --oneline HEAD..origin/main
0381fe0 fix(emulator): RunCanary's resumed-activity detection is
stale for API 34+
7fa49f7 feat(emulator): add containerized-runner support to
RunCanary/emulator-canary
```

boba’s pin is **2 commits behind** origin/main. Both commits are Android-emulator work (pkg/emulator), **unrelated** to pkg/monitor/pkg/policy/pkg/health/ pkg/boot/pkg/compose — the packages relevant to this task. **No blocking upstream-sync need for Phase 2.** A routine git submodule update --remote + pointer bump is still owed at some point per normal hygiene, but is not a Phase 2 prerequisite.

1.2 What it exposes

Top-level layout (find . -maxdepth 2, verbatim, abbreviated to the load-bearing paths):

```
cmd/{applesim,boot,ctop,deploy-stack,distributed-
build,distributed-test,
    emulator-canary,emulator-cleanup,emulator-matrix,genymotion,
    ota-device-emu-boot,vm-matrix}/          # 12 single-file
(`main.go`) binaries
pkg/
{applesim,boot,brokertest,cache,compose,crossbuild,ctop,cuttlefish,
discovery,distribution,egress,emulator,endpoint,envconfig,event,
health,i18n,lazyservice,lifecycle,logging,metrics,monitor,network,
    orchestrator,policy,remote,remoteexec,runtime,scheduler,
    serviceregistry,vm,volume}/              # 30 packages
internal/{buildpkg,exec,netaddr,platform}/
scripts/{host-power-management,resource-policy,anti-bluff,lib}/
tests/{benchmark,configs,e2e,integration,security,stress}/
challenges/{scripts,baselines}/
docs/ (20 .md files, each with .html + .pdf twins per §11.4.65)
```

Package purposes directly relevant to a watchdog (from CLAUDE.md’s own table, cross-checked against source):

Package	Relevance to watchdog
	Primary substrate.
pkg/monitor	ResourceMonitor interface: periodic poll → ResourceSnapshot → ThresholdEvaluator → fire Action func(*ResourceSnapshot). Already a working poll-evaluate-act loop (§3.5).
pkg/policy	Resource-cap policy (mem_limit/ memswap_limit/pids_limit/ oom_score_adj) — the Go counterpart of scripts/resource- policy/policy.yaml, which is the literal mechanism docs/ RESOURCE_LIMITS.md documents as “ Layer 1 ” of the same 3-layer defense-in-depth this task is extending.
pkg/health	

Package	Relevance to watchdog
	HealthChecker (TCP/HTTP/gRPC/Custom) — reusable if the watchdog exposes its own liveness endpoint (recommended, §5 gap analysis).
pkg/boot	BootManager — the sanctioned orchestration entry point (Hard Stop #3 / §11.4.76) if the watchdog is ever containerized.
pkg/compose	Compose file orchestration — relevant only under a container-shaped option (D).
pkg/lifecycle	IdleShutdown, lease/semaphore primitives — pattern reference for “self-manage own resource footprint,” not directly reusable (the watchdog must NOT idle-shutdown; it is the opposite of an idle-managed service).
pkg/event	Pub/sub event bus (20 event types) — a plausible internal notification channel if the watchdog is built as a Go binary using this module’s own primitives.
pkg/metrics	Prometheus-compatible metrics — directly reusable for “performance tests: watchdog overhead” evidence (§4).
pkg/runtime	Container-runtime abstraction (Docker/Podman/K8s auto-detect) — only relevant under Option D.

1.3 Build/test tooling

```

build / test / test-race / test-short / test-integration / test-
    bench /
test-coverage / fmt / vet / lint / clean / challenge /
anti-bluff-scan / anti-bluff-anchors / anti-bluff-mutation /
anti-bluff-mutation-changed / anti-bluff / update-baseline / qa-
    all / help

```

Go 1.25.0. go.mod deps: prometheus/client_golang, stretchr/testify, golang.org/x/crypto, golang.org/x/term, gopkg.in/yaml.v3 — all third-party, none own-org.

Mutation testing is real, not aspirational: .go-mutesting.yml configures the Avito go-mutesting fork with json_output: true, consumed by challenges/scripts/mutation_ratchet_challenge.sh. This is exactly the mechanical §1.1-paired-mutation infrastructure the operator’s anti-bluff mandate demands — it already exists and works for this submodule; a new watchdog package would be covered by the SAME make anti-bluff pipeline with zero new tooling.

docs/test-coverage.md (Revision 1, round 299) is a **symbol → test-type ledger** satisfying CONST-050(B): “is pkg/<X>.<Func> covered by unit + integration + e2e + Challenge + paired mutation?” A new watchdog package inherits this discipline — Phase 2 must add its own ledger rows, not invent a new coverage-tracking scheme.

1.4 Existing test-type coverage (verbatim tree)

```
tests/benchmark/scheduler_bench_test.go
tests/e2e/remote_e2e_test.go
tests/integration/{apple_container_integration_test.go,
                  distribution_integration_test.go,
                  remote_deployment_test.go}
tests/security/ssh_security_test.go          (//go:build
security)
tests/stress/distribution_stress_test.go
challenges/scripts/
{anchor_manifest,apple_container_linux,bluff_scanner,
  chaos_failure_injection,containers_describe,ddos_health_flood,
  host_no_auto_suspend,mutation_ratchet,no_suspend_calls,

scaling_horizontal,stress_sustained_load,ui_terminal_interaction,
  ux_end_to_end_flow}_challenge.sh
```

Unit tests are colocated per-package (pkg/<x>/*_test.go, standard Go convention) — e.g. pkg/monitor has 9 test files, pkg/policy 4, pkg/health 13, pkg/lifecycle 14, pkg/event 6. **Chaos+stress Go test files currently exist only for pkg/cuttlefish** (chaos_test.go, stress_test.go) — pkg/monitor and pkg/policy have **zero** chaos/stress Go test files today (covered only indirectly, at the whole-submodule level, by the shell-script Challenges). This is a real, named gap (§5) if the watchdog logic lands as new code in pkg/monitor.

Test types present at the submodule level, mapped to the operator's requested 14-type list (§4 below has the full per-type plan): unit , integration (-tags=integration), e2e , security (-tags=security), stress (Go + Challenge), chaos (Challenge-level only, no package-level Go chaos tests for monitor/policy), DDoS (ddos_health_flood_challenge.sh), scaling (scaling_horizontal_challenge.sh), benchmark , performance (folded into benchmark, no dedicated overhead-budget assertion), UI/UX (ctop TUI — N/A for a headless watchdog), Challenges , HelixQA (no HelixQA banks exist inside this submodule at all — find . -iname '*helixqa*' returns nothing; HelixQA integration is entirely a **consumer-side** (boba's own submodules/helixqa) concern, confirmed in §6).

2. Dependency audit

2.1 helix-deps.yaml (verbatim)

```
# helix-deps.yaml – Containers submodule Submodule-Dependency
Manifest
# Per HelixConstitution §11.4.31 (Submodule-Dependency-Manifest
Mandate).
#
# Containers is a leaf Go submodule with ZERO own-org submodule
dependencies.
#
# Schema reference: <constitution-submodule>/Constitution.md
§11.4.31.
# Last updated: 2026-05-15 (Phase 3-debt closure for Containers).

schema_version: 1
deps: []

transitive_handling:
  recursive: true
  conflict_resolution: operator-required

language_specific_subtree: false
```

Zero own-org dependencies. Containers is a leaf submodule (§11.4.28(C)-compliant — no nested own-org chain). There is nothing to “update to latest” or “extend upstream” at the *submodule-dependency* layer for this task — §11.4.74’s extend-don’t-reimplement obligation applies entirely **within** this submodule (extend pkg/monitor/pkg/policy,

don't stand up a parallel detection mechanism in boba's own tree) and **within** boba (extend scripts/systemd/user/* + challenges/scripts/resource_pressure_signature_challenge.sh, don't reinvent).

2.2 Third-party Go deps (go.mod)

```
github.com/prometheus/client_golang v1.23.2
github.com/prometheus/client_model v0.6.2
github.com/stretchr/testify v1.11.1
golang.org/x/crypto v0.52.0
golang.org/x/term v0.43.0
gopkg.in/yaml.v3 v3.0.1
```

None of these are watchdog-blocking; prometheus/client_golang is directly usable for the performance-overhead metric a watchdog should expose (§4/§5). No new third-party dependency is structurally required for a Go-native detector — PSI files and cgroup files are plain-text /sys/fs/cgroup/... reads, no library needed (the existing pkg/monitor/system.go pattern of hand-rolled bufio.Scanner parsing over /proc/stat//proc/meminfo is the established, zero-dependency style to follow).

2.3 Extend-don't-reimplement candidates found

- pkg/monitor.SystemCollector / ResourceSnapshot / ThresholdEvaluator — extend, don't replace (§3.5, §5).
 - pkg/policy.Cap/Policy — the OOM-adj/pids-cap primitive already exists and already matches boba's own docker-compose.yml container-hygiene corollary (§6.2) — a watchdog-specific cap tier can be a new policy.Rule, not new code.
 - challenges/scripts/resource_pressure_signature_challenge.sh (in **boba**, not Containers) — the 5-signature detector already implements 3 of the 4 host-pressure signals a watchdog needs (RSS, thread%, PSI). Reuse this logic (ideally promoted into a Go package under pkg/monitor for testability + mutation coverage, per §5), never re-derive it.
 - scripts/host-power-management/ (in Containers) — a working 3-layer defense-in-depth *pattern* (host-level mask / session-level bootstrap / source-tree static gate) for a **different** CONST-033 concern (suspend prevention), but structurally the exact template to mirror for the watchdog's own defense-in-depth layering.
-

3. Design: the external-watchdog concept (Option-D-shaped, contingent — see §0)

This section is written under the explicit, load-bearing caveat of §0: it describes the only container-shaped design that is not already falsified by this host's own incident forensics — a container run under a *second, dedicated* UID. It is NOT a description of “run today's boba containers' sibling under UID 1000,” which is the premise this audit's dispatch used and which the sibling proposal doc disproves with direct journal evidence. If the operator instead approves Option B (cron) or Option A (system-level systemd unit) per the sibling proposal's recommendation, most of §3.2–§3.4 collapses to “n/a, not a container” and only §3.5 (the detection-logic design) survives unchanged — which is exactly why §3.5 is written to be supervision-mechanism-agnostic.

3.1 What the watchdog does (option-agnostic)

Probe host/target-slice pressure signals every N seconds (default cadence matching the existing timer's 1-hour cycle is too coarse for a “catch it before the kill” mandate — recommend 60s, matching PSI's own avg10 window), evaluate against thresholds, and on crossing: (a) emit a loud, durable alert **outside** `user@1000.service`'s journal/log namespace (§3.6), (b) optionally — gated, never autonomous per §11.4.101/§11.4.252 fail-closed-on-dangerous-combination — signal/kill a specifically-identified runaway process. It does **not** replace `boba-resource-pressure-check.timer`; it is explicitly the backup layer for the one scenario (§12.10-class total pool death) that timer cannot itself observe, mirroring `docs/RESOURCE_LIMITS.md`'s own “Layer 3 — system-side oomd... NOT used on this host because we do not have sudo here” gap — this task is precisely what fills that named, pre-existing gap.

3.2 Where it runs (Option-D framing)

A dedicated, low-privilege system account (e.g. `boba-watchdog`), created once by root (`useradd + loginctl enable-linger boba-watchdog`), running a **rootless** podman container (§11.4.161-compliant — rootless is preserved; only the one-time account provisioning needs root, identical privilege *class* to Option A) under that UID's own `user@<uid2>.service` — structurally independent of `user.slice/` `user-1000.slice` by construction, per the sibling proposal's own cgroup analysis (§2.4 of that doc). Orchestrated through this Containers submodule's `pkg/boot.BootManager`, never raw `podman run`, per Hard

Stop #3 / §11.4.76 — the watchdog’s own compose file lives outside boba’s main docker-compose.yml (a separate docker-compose.watchdog.yml, started by a watchdog-specific wrapper analogous to start.sh, never folded into the existing boba.target/boba-svc.sh lifecycle, since that lifecycle is itself scoped to user@1000.service).

3.3 How it observes user.slice (Option-D framing)

Mount **read-only**: /sys/fs/cgroup/user.slice (for PSI + pids.current/max + memory.current/max under user-1000.slice) and /proc (for per-process RSS / cmdline scanning, SIG-1/SIG-5-equivalent). Podman’s default rootless netns/userns mapping does not by itself grant read access to another UID’s cgroup files — **this is an unverified, load-bearing assumption requiring a live Phase-1.5-class spike** (mirroring the sibling proposal’s own explicitly-flagged Phase-1.5 need for Option B): confirm on this actual host whether boba-watchdog’s container, with /sys/fs/cgroup/user.slice bind-mounted read-only, can actually cat user-1000.slice/memory.pressure and user-1000.slice/pids.current — cgroupfs read permissions are typically world-readable on most distros but this is ALT-Linux-specific and **not yet measured** (§11.6 honest gap). SIG-3-equivalent (podman container-log EAGAIN scanning) requires the watchdog to reach UID 1000’s rootless podman socket — rootless podman sockets are owned 0700 by the running UID by default; **this signal is NOT reachable from a second-UID container without an explicit, additional privilege grant this design does not currently propose** — recommend dropping SIG-3-equivalent from the external watchdog and leaving it exclusively to the in-slice timer (which already has it), rather than widening the watchdog’s privilege footprint to chase a signal explicitly described in the existing script’s own comments as “correlation-as-context,” not a primary independent signal.

3.4 How it emits alerts (option-agnostic)

Durable log path OUTSIDE user@1000.service’s journal namespace: system journal (journalctl with no --user, reachable from system.slice/a second-UID user@<uid2>.service alike) plus a plain append-only file under a path owned by the watchdog account (e.g. /var/log/boba-watchdog/ or ~boba-watchdog/alerts.log), readable by the operator’s own account via group membership — **cross-account**

notification design is explicitly flagged as undesigned in the sibling proposal's own §8 open-items list; this audit concurs it is a real, non-trivial Phase 2 design item, not a detail to hand-wave.

3.5 Integration with the existing timer (option-agnostic — the actual reusable core)

Defense-in-depth, not replacement: `boba-resource-pressure-check.timer` keeps running (richer `journalctl --user -u visibility`, all 5 signatures including SIG-3, while the pool is alive); the watchdog is the layer that survives the one scenario the timer structurally cannot. **The detection logic itself — PSI threshold, RSS threshold, thread%/pids-ratio threshold — should be the SAME logic in both places**, invoked two different ways:

- Today: bash, self-\$USER-scoped (`challenges/scripts/resource_pressure_signature_challenge.sh`).
- Needed: the same three signals (SIG-1/2/4-equivalent), re-parameterized to accept an explicit **target** (UID, or cgroup path) rather than assuming “the invoking process’s own scope” — a mechanical generalization, not new detection science. `pkg/monitor.ThresholdEvaluator/resolveMetric` (§5) is the natural Go home for this generalized, testable, mutation-covered form; the bash script becomes a thin wrapper (or is retired in favor of a small Go CLI binary under `cmd/`, matching this submodule’s existing `cmd/ctop-style` convention) once the Go form exists.

3.6 Failure mode if the watchdog itself dies

Under Option D: `Restart=on-failure` in the watchdog’s own `systemd-user` unit (inside `user@<uid2>.service`, which is NOT the failure-prone pool) — standard `systemd` restart-policy recovery, the same mechanism `boba-resource-pressure-check.service` already deliberately does NOT use (\$ comment in that unit file: “a oneshot’s ‘failed’ status is itself the alert... auto-restarting the service would duplicate/race the timer’s own schedule”) — but the watchdog, unlike the existing oneshot timer, is a **long-running poll loop** (§3.1), so `Restart= on-failure + OnFailure= alerting` is the correct, standard pattern here, not a duplicate of the oneshot’s design. Under Option A/B (the recommended paths): `Restart=on-failure` on the system unit (A), or a monitoring cron-of-crons pattern / `crond`’s own liveness (B) — `crond.service` itself failing is a pre-existing, orthogonal host-health question outside this task’s scope.

4. Test coverage plan (§11.4.27, all 14 requested types)

Applies regardless of which option (A/B/D) Phase 2 lands, since the detection *logic* (§3.5) is shared. Container-specific rows are marked **[D-only]**.

Type	Plan	Status today
Unit	Mock /proc//sys/fs/cgroup reads via injected file paths, mirroring pkg/monitor/system.go's existing collectCPULinuxFromFileOK(path) testability pattern (table-driven, testify). Cover: PSI-line parsing, pids-ratio math, threshold evaluation, target-UID parameterization.	Pattern exists in pkg/monitor; zero PSI/cgroup-specific unit tests exist yet (gap, §5).
Integration	Real reads against a real, but test-owned, cgroup scope (systemd-run --user --scope a throwaway process, read its actual pids.current/memory.current) — -tags=integration, matching tests/integration/ convention.	New — no existing integration test target cgroup files.
E2E	Spawn a fixture that genuinely IS pressure (bounded, host-safety-respecting per §12.6/§12.9 — e.g. a stress-ng --vm 1 --vm-bytes 200M --timeout 5s inside a disposable cgroup scope with a hard MemoryMax), verify the detector's PSI/RSS reading crosses threshold and fires. Must itself respect the 30-40% host-resource-budget mandate (Universal Mandatory Constraint #9) — bound the fixture's own cap tightly.	New. challenges/fixes: resource_pressure/sig4_seeded_psi_fixture (boba-side) already establishes the “seeded” fixture PSI file rather than induce real pressure; pattern for the threshold comparison half — need to add a genuinely-live fixture only for the full poll e2e path.
Full-automation	systemd/container lifecycle: install → enable → simulate kill of the watched pool → confirm watchdog fires anyway. [D-only, or A/B-equivalent under those options] — this is exactly the Phase-1.5 live spike both this doc (§3.3) and the sibling proposal (§3, Option B “Cons”) already flag as a hard, unskippable prerequisite before ANY option is trusted.	Not yet run for any cgroup
Security	Least-privilege verification: watchdog process/container has read-only mounts only, no write access to /proc//sys/fs/cgroup, no capability	New — no existing test covers this new attack surface.

Type	Plan	Status today
	beyond what reading PSI/pids files requires (none — plain file read), the optional kill-action path (§3.1) is gated per §11.4.252 fail-closed (refuse unless the target PID’s identity is independently re-verified immediately before signaling — a classic TOCTOU risk for any “watch then kill” design). -tags=security, matching tests/security/ convention.	
DDoS	Watchdog resists a flood of its own trigger condition (e.g., PSI oscillating across threshold every poll) without itself becoming a resource problem — rate-limit repeat-alert emission (dedup/backoff), matching the ddos_health_flood_challenge.sh shape (Challenge, not necessarily Go).	New.
Scaling	Multiple watched slices/UIDs (forward-looking — this host is currently single-operator, but the primitive should not hardcode “exactly one UID”), matching scaling_horizontal_challenge.sh shape.	New; design already parameterizes by tag (§3.5), so scaling is a surface test, not new
Chaos	Kill the watchdog mid-scan, verify systemd/cron restart recovers cleanly with no stuck lock/partial-write (mirrors this project’s own §11.4.85 mandate + the pattern in pkg/cuttlefish/chaos_test.go, the ONE existing chaos Go test in this submodule).	New for pkg/monitor existing pkg/cuttlefish/chaos_test.go is the style reference.
Stress	Long-running under sustained load (hours, not seconds) — no goroutine leak, no fd leak, no unbounded memory growth in the poll loop itself, matching tests/stress/distribution_stress_test.go + stress_sustained_load_challenge.sh conventions.	New for pkg/monitor conventions exist.
Performance	Watchdog overhead <1% CPU, bounded RSS — assert via pkg/metrics (Prometheus client already a dependency) self-instrumentation, or /proc/self/stat self-read, with a hard CI-asserted ceiling (§11.4.24-class build/runtime-resource tracking, “Heisenberg-class observer constraint” — mirrors the existing §11.4.24 sampler’s own “<50MB RSS, <5% CPU” self-limit precedent).	New — no existing overhead-budget assertions anywhere in this submodule.

Type	Plan	Status today
Benchmark	go test -bench=. for the PSI-file-parse + threshold-eval hot path, matching tests/benchmark/scheduler_bench_test.go convention.	New; convention ex
UI/UX	N/A — the watchdog is headless (confirmed: task brief itself notes N/A if no UI; pkg/ctop is this submodule’s only TUI surface and is unrelated).	N/A — honestly, not
Challenges	New challenges/scripts/watchdog_survives_user_slice_kill_challenge.sh (boba-side, or Containers-side if the detection package lands there) — the single most important anti-bluff proof: literally reproduce a user@1000.service-scope SIGKILL against a throwaway equivalent scope and show the watchdog process is unaffected + still fires. This IS the e2e/full-automation test above, packaged as a Challenge per this project’s convention.	New; this is the load bearing anti-bluff an the whole task exists produce.
HelixQA	Add a bank entry in boba’s own submodules/helixqa (confirmed §1.4: HelixQA has zero presence inside the Containers submodule itself — it is purely a consumer-side integration point).	New, boba-side only

5. Gap analysis

5.1 In the Containers submodule

1. **No PSI (pressure-stall-information) reading capability anywhere in pkg/monitor.** Confirmed by direct grep: the only “cgroup”/“pressure” hits in pkg/monitor/system.go are incidental comment mentions about /proc/stat counter edge cases under CPU hotplug/namespace changes — not actual PSI-file parsing. **This is the single largest capability gap.**
2. **No cgroup-file reading at all** — pkg/monitor.SystemCollector reads /proc/stat (whole-host CPU) and /proc/meminfo (whole-host memory) only; there is no concept of a *scoped* (per-slice, per-UID) resource view anywhere in this package.
3. **pkg/monitor.ThresholdEvaluator.resolveMetric** (pkg/monitor/threshold.go:66-98) supports exactly two metric namespaces — system.* (whole host) and container.<name>.* (via the container runtime) — with **no slice.<name>.* or cgroup.<path>.***

namespace. Adding one is a small, mechanical, additive change (a new case arm), not a redesign.

4. **Zero chaos/stress Go tests for pkg/monitor or pkg/policy** (only pkg/cuttlefish has them today) — any new watchdog-detection code landing in pkg/monitor inherits this submodule’s general test-type discipline but starts from zero package-specific chaos/stress coverage, not from an existing baseline.
5. **No existing “kill a specifically-identified process” primitive anywhere in this submodule** — pkg/policy sets oom_score_adj (an OOM-killer *preference*, not a direct kill), and nothing else in the audited packages sends a signal to an arbitrary PID. If Phase 2 approves the optional-kill capability (§3.1), this is wholly new, high-blast-radius code requiring the §11.4.252 fail-closed-on-dangerous-combination discipline from first principles — not an extension of anything that exists.
6. **No HTTP/health-endpoint scaffolding wired up for a new standalone binary** by default — pkg/health.HealthChecker exists and is reusable, but a new cmd/watchdog/main.go would need to wire it explicitly (small, well-trodden pattern per cmd/boot/main.go’s existing style — not read in this audit’s time budget, flagged as a Phase 2 read-first item).

5.2 In its dependencies

None found requiring update or extension. §2 confirms zero own-org dependencies (leaf submodule) and no third-party dependency gap — PSI/cgroup files are plain-text reads needing no new library, matching this submodule’s existing zero-dependency /proc parsing style.

5.3 Existing prior art to extend (§11.4.74), summarized

- pkg/monitor poll-evaluate-act loop (DefaultMonitor.Start/collect, pkg/monitor/monitor.go:58-163) — extend with a second collector for slice-scoped metrics, reusing the SAME ThresholdEvaluator/ResourceSnapshot shape rather than a parallel mechanism.
- challenges/scripts/resource_pressure_signature_challenge.sh (boba-side) — the actual 5-signature detection *science* already exists and is already-forensically-tuned to this exact host’s real incident thresholds (SIG1_MAX_PROC_RSS_GB=5, SIG2_THREAD_PCT=70, SIG4_PSI_AVG60_LIMIT=50) — reuse the thresholds and the parsing logic, generalize the *scope* (§3.5).
- pkg/policy — the OOM-preference cap mechanism, directly reusable if the watchdog’s own container (Option D) needs a cap

tier, and conceptually the right place to record “the watchdog itself must never be OOM-preferred over the pool it protects” as an explicit, tested policy statement.

- scripts/host-power-management/ 3-layer defense-in-depth pattern (host mask / session bootstrap / source-tree gate) — structural template, not code, for organizing the watchdog’s own layered rollout.

5.4 Estimated implementation time (ESTIMATE — needs live verification, §11.4.6)

These are rough estimates only, consistent with (not a re-derivation of) the sibling proposal’s own §6 cost table, which already covers Options A/B/D at the *host-mechanism* layer. This audit adds the *Containers-submodule* layer on top:

Work item	Estimate
PSI/cgroup-file reader + slice.* metric namespace in pkg/monitor (unit-tested)	~2–4 hours
Target-UID parameterization of the existing 3 relevant signatures (SIG-1/2/4-equivalent)	~1–2 hours (mostly re-plumbing existing logic)
Integration + e2e + chaos + stress Go tests for the above, to this submodule’s existing bar (mutation-covered, ledger- tracked)	~4–8 hours
Phase-1.5 live spike (whichever option — cron-cgroup check per sibling doc, or podman-cross- UID-cgroup-read check per §3.3)	~15–60 min, hard blocking prerequisite, not skippable
Full Option-D container wiring (pkg/boot/pkg/compose, second UID provisioning, cross-account alerting) — only if Option D is chosen over B/A	~1 day+ (unchanged from sibling doc’s own estimate)
Option A/B wiring (systemd unit or cron entry reusing generalized detector)	~1–2 hours (unchanged from sibling doc’s own estimate)

6. Cross-submodule impact

- **docker-compose.yml**: Not the primary integration point under Option D either — boba's existing compose file's services (qbittorrent, jaxxnet, qbittorrent-proxy-go, download-proxy, boba-jaxxnet) all already carry the `mem_limit/pids_limit/oom_score_adj: 500` container-hygiene corollary (verified: `grep -n "mem_limit\|oom_score_adj\|pids_limit" docker-compose.yml` hits at lines 27-30, 50-53, 119-122, 204+ — consistent with the pattern docs/RESOURCE_LIMITS.md/pkg/policy document) — but every one of those services runs *inside* `user@1000.service`, i.e. inside the exact pool a watchdog must be outside of. A watchdog service does **not** belong in this file under any of options A/B/D; under D it needs its own, separate compose file, deliberately not wired into `boba.target`.
 - **scripts/**: yes, under any option — a new `scripts/install-watchdog-*.sh` (mirroring `scripts/install-resource-pressure-timer.sh`'s existing mechanical- verification pattern: symlink-not-copy unit sources, `systemctl/crontab list-and-verify`, captured evidence under `docs/qa/task-85-*/`) is needed regardless of A/B/D.
 - **scripts/systemd/user/***: the EXISTING `boba-resource-pressure-check`. `{service,timer}` stays as-is (kept running per §3.5's defense-in-depth framing) — no change needed there for this task, only additive new units/ `crontab` entries alongside it.
 - **constitution**: No new anchor needed. §11.4.85 (stress+chaos mandate) already covers this class of requirement; §11.4.252 (fail-closed-on-dangerous- combination, 2026-08-15) already covers the optional-kill-action safety discipline; §12.12 (RLIMIT_NPROC awareness) and §12.6 (memory ceiling) already cover the host-safety bounds any watchdog fixture/e2e-test must respect; §11.4.161 (rootless mandate) already scopes what Option D would need to justify. This is a case of *applying* existing anchors, not minting a new one.
 - **submodules/helixqa**: yes — a new HelixQA bank entry is owed (§4, HelixQA row) once Phase 2 lands, boba-side only (confirmed §1.4: zero HelixQA presence inside the Containers submodule itself).
 - **submodules/containers pointer bump**: only if Phase 2 chooses to land the new `pkg/monitor` capability in the submodule (recommended) rather than entirely in boba's own tree — in which case the normal §11.4.26 constitution- submodule-adjacent-but-not-identical workflow applies (fetch/pull first for Containers per §11.4.37, commit+push to Containers' own upstreams, then bump boba's pointer) — this is standard submodule hygiene, not a special case.
-

7. Anti-bluff evidence log

All command output above is verbatim from this session, run against the real checked-out submodule tree — no summarization of findings, no elided negative results. Specifically preserved verbatim: `git submodule status`, `git fetch + git log HEAD..origin/main` (upstream drift), `helix-deps.yaml` full contents, `pkg/monitor/threshold.go` `metric-namespace switch` (proving the `slice.* gap` by absence), `pkg/policy/policy.go` `Default()` `cap table` (proving the existing OOM-preference mechanism), `docs/RESOURCE_LIMITS.md` §1-2 (proving “Layer 3... NOT used... no sudo” is a pre-existing, named, unfilled gap this task closes), the sibling proposal doc’s own incident-log excerpt (proving Option C-as-framed is dead), and the `docs/workable_items.db` query result for BOB-121 (proving no operator approval is recorded for the container-under-UID-1000 framing this audit was dispatched with).

8. Recommended Phase 2 dispatch plan (contingent on §0 reconciliation)

Blocking, before ANY Phase 2 code dispatch:

1. **Operator reconciliation of §0** — confirm explicitly whether the intended Phase 2 target is (a) Option B (cron, sibling-doc-recommended, near-zero cost, no new root infra), (b) Option A (system-level systemd unit, one-time sudo, escalation path if B’s Phase-1.5 spike is adverse), or (c) Option D (second-UID rootless container, heaviest, “only revisit if a future incident shows the kill reaching beyond UID 1000’s slice” per the sibling doc’s own words) — i.e. genuinely override the sibling doc’s recommendation. **This single decision determines nearly the entire shape of Phase 2** and must not be inferred or guessed (§11.4.6).
2. **Phase 1.5 live spike** (named as hard-blocking by BOTH this doc and the sibling proposal, for whichever option is chosen): for B, confirm a cron job’s `/proc/<pid>/cgroup` is genuinely outside `user@1000.service`; for D, confirm a second-UID container can actually read `user-1000.slice`’s `PSI/pids cgroup` files with only a read-only bind-mount (§3.3) — **neither is verified yet, by anyone, for this exact host.**

Once (1)+(2) resolve, suggested dispatch (independent of A/B/D choice for the first two items):

1. Generalize the 3 relevant signatures (SIG-1/2/4-equivalent) to accept an explicit target UID/cgroup-path instead of self-\$USER scoping — either as a boba-side script change, or promoted into pkg/monitor as a new Go package with unit+integration+chaos+stress coverage per §4 (recommended, for the mutation-testing + coverage-ledger discipline this submodule already provides for free).
2. Wire the chosen supervision mechanism (cron entry / systemd unit / Option-D container+second-UID) per §3.2–§3.4, using this submodule's pkg/boot only if Option D.
3. Author the Challenge that reproduces a `user@1000.service-class SIGKILL` against a disposable equivalent scope and proves the new mechanism survives it and fires — this is the actual anti-bluff proof the whole task exists to produce; nothing else in this plan matters if this Challenge cannot be written and does not genuinely fail-then-pass (§11.4.115 RED-then-GREEN).
4. Add the HelixQA bank entry (boba-side).
5. Update docs/test-coverage.md-equivalent ledger rows.